

Executive Summary

We propose transitioning from simple Euclidean routing to a hybrid matrix-based **insertion + local-search** solution. Key recommendations: use a road-network API (self-hosted OSRM or GraphHopper) for **true travel times/distances**, and assign each new delivery to the driver with the lowest *marginal route-cost increase*. Then optimize that driver's route via a nearest/cheapest insertion heuristic followed by 2-opt improvements. This balances speed, quality, and engineering effort. In short: **Phase 1** – quick fixes (Haversine or OSRM-based distances, better assignment rule, UI bugfixes). **Phase 2** – integrate a distance matrix service and replace brute-force with insertion+2-opt. **Phase 3** – optional full VRP solver (e.g. OR-Tools) with capacity and time-window constraints for complete optimization. Each phase yields large gains (lower total travel time and lateness) for modest effort.

Key elements (details below):

- **Problem Dimensions:** If unspecified, mark them accordingly (e.g. “time windows: unspecified, assume none”). Likely stops/route $\approx 5\text{--}20$, drivers dozens, dynamic arrivals yes, SLAs moderate.
- **Algorithmic Approach:** Exact Held-Karp (DP) only for *very* small routes (≤ 15 stops ¹). Otherwise use **heuristics**: greedy insertions + 2-opt local search. These give near-optimal results fast ² ³. For multiple drivers, do *sequential insertion* (try every driver and pick best insertion) rather than round-robin.
- **Agent Selection:** Compute the cost increase (marginal cost) of inserting a new order into each driver's route; assign to the minimum (instead of just “fewest active deliveries”). This simple rule greatly improves proximity and balance ².
- **Travel Matrix Integration:** Use **OSRM** or **GraphHopper** (open-source, self-hosted) for large, fast matrix queries ⁴ ⁵. Mapbox is easy but limited (25×25 per request, billed by elements ⁶). Self-hosting avoids per-call costs and handles bigger jobs. Cache pairwise distances/time in-memory with TTL or by region/time window to minimize API calls (GraphHopper already caches shortest-path trees ⁵).
- **Local Search:** After insertion, run 2-opt swaps until no improvement. 2-opt has worst-case $\sim O(n^3)$ ³ but is fast for $n \sim 20$. It dramatically removes route inefficiencies and nearly matches the quality of sophisticated solvers.
- **Cost Function:** Use a weighted sum, e.g. $J = 0.55 \cdot T_{\text{travel}} + 0.20 \cdot D + 0.15 \cdot P_{\text{late}} + 0.05 \cdot L_{\text{load}} + 0.05 \cdot D_{\text{detour}}$, where each term is (travel time, distance, lateness penalty, load imbalance, pickup detour). Tune weights to business goals (fast delivery vs fuel economy).
- **Concurrency/Safety:** Use MongoDB atomic updates or transactions to move orders between “pending” and “active” safely ⁷. Avoid race conditions by updating a single agent document at a time (findOneAndUpdate) or using transactions for multi-doc moves.
- **Frontend Fixes:** Ensure driver availability logic checks both `activeDeliveries` and `pendingPickupDeliveries` (currently all show available) and map orders by consistent ID. Persist assignment actions through the backend API (not just local UI).

Below is a detailed analysis, a comparative algorithm table, pseudocode, charts, and a phased roadmap. All claims are cited from routing literature and APIs (OSRM/GraphHopper/Mapbox docs, OR-Tools guides, etc.).

Problem Dimensions

- **Stops per Route:** Unspecified (likely ~5–20 on a delivery run). This affects algorithm choice: exact TSP (Held–Karp) only feasible for *very small* n (≤ 15 ¹); beyond that use heuristics.
- **Fleet Size:** Unspecified (assume tens of drivers). An insertion heuristic scales easily to dozens; large fleets can later use VRP solvers if needed.
- **Dynamic Arrivals: Yes** – orders come in real-time. This favors *online insertion heuristics* (assign on arrival) over batch-solver.
- **Time Windows / SLAs:** Unspecified. Assume orders have soft deadlines (desired ETAs). We include a lateness penalty in cost. If hard windows exist, they can be added later.
- **Service Times / Capacity:** Unspecified. We assume each stop is a small delivery (light cargo, short service), and vehicles have adequate capacity (or each carries one delivery at a time). If later needed, capacity can be added as a constraint in VRP.
- **Traffic Variability:** Unspecified. Phase 1/2 assume static “typical” drive times from the matrix API. Phase 3+ could use time-dependent data (Mapbox ‘driving-traffic’ profile) if important.

(Each “unspecified” parameter is noted above. We design a flexible solution, adding constraints only when given.)

Algorithmic Approaches

Single-Vehicle Routing (TSP)

- **Exact DP (Held–Karp):** Guarantees optimal tour by dynamic programming, but $\Theta(2^n n^2)$ time ¹. Practically, use only for *very small* stop counts (e.g. $n \leq 12$ –15). Example: $n=15 \Rightarrow \sim 2^{15} \times 225 \approx 7$ million ops, doable in optimized code. For larger n , it explodes.
- **Nearest Neighbor / Greedy:** Start at depot, repeatedly go to nearest unvisited stop. Runs in $O(n^2)$. Very fast but tour quality is poor (often 10–30% longer than optimal) ⁸. We cite this only as a baseline; it’s too crude for our needs.
- **Insertion Heuristics:** Build a tour by inserting stops one by one. Variants: *Cheapest Insertion* (each new stop goes where it least increases distance) and *Nearest Insertion*. Complexity $\sim O(n^3)$. These give *much* better tours than naive greedy. Importantly, they work naturally in a dynamic setting: start with an empty route and insert as orders arrive ². As one source notes, insertion heuristics are attractive for online routing (despite being suboptimal) ².
- **Local Search (2-opt/3-opt):** Once a tour exists, apply local improvements. **2-opt** swaps two edges to remove crossings: loop over all pairs (i,j) and reverse the segment if it shortens the route ³. This is worst-case $O(n^3)$ (nested loops + rebuild cost) ³, but runs quickly for $n \sim 20$. It typically yields near-optimal routes. **3-opt** tries all 3-edge exchanges ($O(n^4)$ worst-case) and gives even smaller gains at much higher cost, so we usually stick with 2-opt. Lin–Kernighan (LKH) is state-of-the-art for TSP but hard to implement. In practice, insertion+2-opt suffices for quality.
- **Metaheuristics (OR-Tools):** Google OR-Tools supports sophisticated local search (Guided Local Search, Simulated Annealing, Tabu, etc ⁹). These can be used as “tune-ups” if needed, but even OR-Tools uses insertion+2-opt as a first step (e.g. `PARALLEL_CHEAPEST_INSERTION`) ¹⁰.

Recommendation (single-driver): Use a fast heuristic. For up to ~50 stops, do a cheapest-insertion followed by 2-opt (with maybe a short time limit if needed). This yields near-optimal tours with minimal code. Held–Karp is only for tiny routes.

Multi-Vehicle Routing (VRP)

- **Sequential Insertion (Online VRP):** When each new order arrives, attempt to insert it into each active driver's route and pick the best fit. This is standard in dynamic VRP research ². We do *not* simply load-balance by count; we consider route distance. One can use the “sum of squares” or other fairness criteria, but simplest is to minimize added travel cost.
- **Clarke-Wright Savings:** A classic batch VRP heuristic. Could be used in occasional full-reopt passes (Phase 4/A-B). Not needed for real-time assignment.
- **Google OR-Tools (Batch VRP):** For a full re-optimization (offline or major re-route), OR-Tools' Vehicle Routing can handle capacity, time windows, etc ¹¹. It finds high-quality solutions but requires building a model and could take seconds for tens of stops (though optimized C++ is fast). Plan this for Phase 3 or as an overnight process.
- **Load Balancing:** We can optionally add workload fairness by penalizing assignment to heavily-loaded drivers. The insertion cost function could include a term like current active count.

Recommendation (multi-driver): Use the marginal-insertion rule described below. Only after validating gains should we consider heavy-duty VRP solvers.

Marginal-Insertion Agent Selection

To assign a new delivery, we compute each driver's **marginal cost** of adding that delivery to their route:

```
oldCost = cost(driver.route)
bestInsertCost = ∞
for pos in 0..len(driver.route):
    newRoute = insert(delivery, position=pos, into=driver.route)
    c = cost(newRoute)
    bestInsertCost = min(bestInsertCost, c)
marginalCost = bestInsertCost - oldCost
```

Pick the driver with lowest marginalCost (breaking ties by least total load). This directly optimizes travel/time increase, rather than ignoring location. Implementation is $O(m^2)$ per driver (m stops $\times$ cost calc). Since m is small (≈ 20), this is fast. This method is justified in the literature: insertion heuristics with cost-based selection outperform naive rules ².

If no driver can feasibly insert (e.g. capacity or time-window violations), the order waits or triggers hiring a backup driver.

Travel-Matrix Integration (OSRM/GraphHopper/Mapbox)

Instead of Euclidean, use real-road distances and times. Options:

- **OSRM (Open Source Routing Machine):** Free, open-source, self-hosted. High-performance C++ engine for OSM data. Supports a **Table service** to compute full matrices of travel *times* and *distances* ¹². Example: GraphHopper's blog cites OSRM-like speeds – 1000 $\times$ 1000 matrix in ~ 8 s (180k distances/sec) ⁵. With OSRM you pay only hosting costs; no per-call fees ⁴.

- **GraphHopper:** Another open routing engine. Their cloud Matrix API also does huge batches (see chart in 5). The open-source GraphHopper engine can run locally similarly. It's slightly slower than OSRM but still very fast on large matrices. It also supports TURN restrictions and other profiles.
- **Mapbox Directions/Matrix API:** Easy drop-in but limited: max 25 coordinates (625 elements) per request for driving (60 req/min) 6. Billed by element beyond free tier. Good if you want turn-by-turn ETA or traffic. But if you need 50+ stops or many calls, costs grow and rates limit you.

Recommendation: Start with a self-hosted solution. OSRM is optimized for speed (Uses Contraction Hierarchies internally 4), making it ideal for large matrices with zero usage fees. GraphHopper is also viable. Use the “driving” profile (no live traffic). Only use Mapbox or other paid API for prototyping or very small-scale.

Caching / Invalidation: Cache matrix results to avoid repeated calls. For example, keep a cache of last N queries (or split by region/time). In a route update we typically only add/remove 1 stop, so many distances persist. GraphHopper's blog shows caching shortest-path trees can speed up matrices 5 – we can adopt a similar strategy: e.g. keep previously computed distance vectors from each stop as partial table. Also, schedule periodic refresh (e.g. every hour) if data could change (traffic or map updates).

Local Search and Cost Function

- **2-opt Optimization:** After building a route, apply 2-opt swaps until no gain. In code, that's nested loops trying all edge-pair reversals 3. This eliminates crossovers. We stop once we go through all pairs without improvement. Empirically, 2-opt yields large improvements quickly. Implementation complexity is moderate: easy to code but $O(n^2)$ loops.
- **Scoring Function:** Define $J = \alpha \times T_{\text{travel}} + \beta \times D + \gamma \times P_{\text{late}} + \delta \times L_{\text{load}} + \epsilon \times D_{\text{detour}}$, where:
 - T_{travel} = total travel time (min)
 - D = total distance (km)
 - P_{late} = lateness penalty (e.g. minutes past SLA * big factor)
 - L_{load} = number of existing assignments for driver (promote balance)
 - D_{detour} = extra time to first pickup vs direct route (optional).

Example weights (tunable): $\alpha=0.55, \beta=0.20, \gamma=0.15, \delta=0.05, \epsilon=0.05$. This matches a priority: minimize drive time most, then fuel (distance), then lateness, fairness, and detour. These can be adjusted (e.g. set γ high if late deliveries are unacceptable).

Use J whenever comparing routes: the insertion selection uses the change in J , and 2-opt tries to lower J .

Complexity and Latency Estimates

- **Marginal Insertion:** Per new order, computing cost for each driver is $O(k \times m^2)$, with k drivers, m stops. E.g. 10 drivers, 20 stops each: $\sim 10 \times (20^2) = 4000$ ops for insertion checks. This is milliseconds in Node/JS.

- **OSRM Matrix:** A 20×20 matrix request takes ~10–50ms on a local server. GraphHopper’s optimized matrix saw 1000×1000 in 8s ⁵, so small ones are trivial.
- **2-opt:** Worst-case $O(m^3)$, but typically converges in few iterations. For $m=20$, even $20^3=8000$ simple ops – a few ms in compiled code.
- **OR-Tools VRP (CP-SAT):** Unknown worst-case, but solving 50 stops + 5 vehicles can take seconds to minutes depending on settings. Useful for nightly or async tasks, not per-order.

In total, expect each delivery assignment to take under 100ms including matrix calls (assuming caching). This is acceptable for real-time updates.

Concurrency and Transactions

- Use **atomic updates** for queue transitions: e.g. `findOneAndUpdate` to move an order from pending to active within one Mongo document if possible ⁷.
- If multiple documents need updating (e.g. `pendingOrders` collection and `driver.collection`), use MongoDB **multi-document transactions** (requires replica set) to ensure either all succeed or none ⁷.
- Alternatively, embed order references inside the driver document so moving from pending to active is a single-doc op.

This ensures no lost or duplicate assignments when, e.g., two orders come in simultaneously.

Frontend/Backend Changes

- **API:** Add endpoints for matrix queries if needed, and for final assignments. The frontend DeliveryManagement should call the server to persist an assignment (not just local state).
- **Driver Map:** After routingService gives the optimized route (list of waypoints), front end (React map) can draw it. OSRM or GraphHopper can provide polylines if integrated. Ensure consistent IDs: the map widget currently mismatches (fix to use `delivery.id`).
- **Live Data:** The dashboard’s `useLiveLogisticsData` must treat a driver as “busy” if either `activeDeliveries` or `pendingPickupDeliveries` is non-empty.
- **Notifications:** When an order is assigned/updated, push the updated ETA to the driver app (via push notification or websockets).

These changes ensure the system uses the new assignment logic and shows routes in the UI.

Comparative Table of Algorithms

Algorithm	Quality	Time Complexity	Stops (practical)	Implementation Effort
Brute Force (n!)	<i>Optimal</i> (exact)	$O(n!)$ (super-exponential)	$n \leq 5$ (tiny)	Very High (impractical)
Held–Karp (DP) ¹	<i>Optimal</i> (exact)	$O(2^n n^2)$ (exponential) ¹	$n \leq 12-15$	High (complex DP code)

Algorithm	Quality	Time Complexity	Stops (practical)	Implementation Effort
Nearest Neighbor	<i>Poor</i> (~20–50% worse) ⁸	$O(n^2)$	Any (small code)	Very Low (trivial)
Cheapest Insertion	<i>Moderate</i> (~5–15% worse)	$O(n^3)$ (heuristic)	≤ 100 (batch)	Low
Insertion + 2-opt ² ³	<i>Good</i> (~near-optimal)	$\sim O(n^3)$ (with 2-opt)	~ 50 (each driver)	Moderate (custom code)
3-opt / LKH	<i>Very Good</i> (close to optimal)	$\sim O(n^4)$ (very heavy)	≤ 30 (very small)	Very High (complex)
OR-Tools VRP ¹⁰	<i>High</i> (handles constraints)	NP-hard (solver-based)	Tens of vehicles $\times$ stops (with time limit)	High (library integration)
OSRM/GraphHopper Matrix ⁴ ⁵	<i>True road times</i>	$\sim 1s$ for 1000×1000 (parallelizable) ⁵	Thousands (batch)	Medium (set up service)

(Rows show solution approaches. “Implementation Effort” reflects how much engineering work: inserting 2-opt is moderate, OR-Tools requires learning its API, etc.)

Key takeaways: heuristics (+2-opt) achieve most gains with moderate coding. Exact solvers (DP, LKH) are impractical beyond tiny routes. OR-Tools is powerful but heavy to integrate. OSRM/GraphHopper give realistic metrics cheaply once set up.

Pseudocode: Insertion + 2-opt (Drop-in)

Below is a sketch of how to modify `routingService.js` and `assignmentService.js`. It assumes a cached matrix `dist[i][j]` of travel times:

```
// assignmentService.js
function assignDelivery(delivery, agents) {
  let bestAgent = null, bestCost = Infinity, bestPos = -1;
  for (const agent of agents) {
    if (agent.status !== 'available') continue;
    const route = agent.activeRoute; // e.g. [start, ..., end]
    const oldCost = routeCost(route);
    // try inserting at each possible position
    for (let pos = 0; pos <= route.length; pos++) {
      const newRoute = route.slice(0,
pos).concat([delivery]).concat(route.slice(pos));
      const cost = routeCost(newRoute);
```

```

        if (cost < bestCost) {
            bestCost = cost;
            bestAgent = agent;
            bestPos = pos;
        }
    }
}
if (!bestAgent) {
    console.warn("No agent could take delivery", delivery.id);
    return;
}
// assign to bestAgent
bestAgent.activeRoute.splice(bestPos, 0, delivery);
// persist in DB (move from pending to active for agent)
saveAssignment(bestAgent.id, delivery.id);
// trigger route re-optimization if needed
}

```

```

// routingService.js
function optimizeRoute(agent) {
    let route = agent.activeRoute;
    let improved = true;
    while (improved) {
        improved = false;
        // 2-opt: try all i<j swaps
        for (let i = 1; i < route.length - 1; i++) {
            for (let j = i + 1; j < route.length; j++) {
                const newRoute = twoOptSwap(route, i, j);
                if (routeCost(newRoute) < routeCost(route)) {
                    route = newRoute;
                    improved = true;
                    break; // restart loops on improvement
                }
            }
            if (improved) break;
        }
    }
    agent.activeRoute = route;
    // update global state if needed
}

function twoOptSwap(route, i, j) {
    // swap path between i and j (inclusive)
    return route.slice(0, i)
        .concat(route.slice(i, j + 1).reverse())
        .concat(route.slice(j + 1));
}

```

```

}

function routeCost(route) {
  // sum of pairwise matrix times (or call to distance API)
  let cost = 0;
  for (let k = 0; k < route.length - 1; k++) {
    cost += dist[route[k].id][route[k+1].id];
  }
  return cost;
}

```

(In practice, ensure `dist` is refreshed from the matrix API whenever stops change. The pseudocode abstracts that detail.)

Benchmark Plan

To evaluate, we will conduct an A/B test on key metrics.

Datasets: Use real or benchmark data, e.g. Solomon/Gehring VRP instances (urban delivery problems) with added dynamic order timing. Or simulate our system with historical location data.

Metrics:

- **Total travel time** (vehicle-minutes): sum of all drivers' times (lower is better).
- **Total distance** (km): fuel efficiency measure.
- **Average ETA error**: difference between predicted vs actual arrival times.
- **Late deliveries**: count/percentage missing their SLA/desired window.
- **CPU time**: compute time per assignment or per re-opt to gauge latency.
- **Load balance**: stddev of deliveries per driver (fairness).

Procedure: Run simulations under “current” (Euclidean + count-based assignment) vs “new” (matrix + insertion/2-opt). Collect metrics over hundreds of orders. Expect the new system to **reduce total time and lateness significantly** (since routes are more direct) at modest compute cost. Also measure per-assignment latency to ensure responsiveness.

Rollout Plan (Phases)

1. **Phase 1 – Quick Wins (2–3 weeks):** Replace straight-line distance with Haversine (if no matrix yet) or basic OSRM calls for key legs ⁴. Implement marginal-insertion for agent selection. Fix frontend bugs (ID mapping, availability). No major refactoring. *Effort: Low–Medium.*
2. **Phase 2 – Heuristic Routing (3–5 weeks):** Deploy OSRM/GraphHopper server and caching. Modify `routingService.js` to use full matrices. Implement insertion+2-opt as above. Update DB schema if needed to store routes. Integrate route polyline on dashboard. *Effort: Medium.*
3. **Phase 3 – Full VRP Solver (optional, 4–6 weeks):** If needed, introduce Google OR-Tools VRP with constraints (capacities, time windows). Use it for nightly reoptimization or large batches. Compare OR-Tools results against Phase 2. *Effort: High (library integration, tuning).*

4. **Phase 4 – A/B Testing & Tuning (2–3 weeks):** Run new vs old in parallel (or via simulation). Adjust scoring weights and thresholds. Deploy gradually and monitor KPIs (late % down, route time down, CPU within SLAs).

```
gantt
    title Implementation Roadmap
    dateFormat YYYY-MM-DD
    axisFormat %b %d
    section Phase 1: Fast Fixes
    Use Haversine or simple matrix      :done,    a1, 2026-05-01, 14d
    Change assignment logic & UI fixes  :done,    a2, after a1, 7d
    section Phase 2: Hybrid Routing
    Set up OSRM/GraphHopper matrix      :active,  a3, 2026-05-20, 14d
    Implement insertion + 2-opt          :         a4, after a3, 14d
    Integrate routes into dashboard     :         a5, after a4, 7d
    section Phase 3: VRP Solver (Opt.)
    Prototype OR-Tools VRP solution     :         a6, 2026-06-20, 21d
    Tune constraints (time windows)     :         a7, after a6, 14d
    section Phase 4: Testing & Tuning
    A/B testing and weight tuning       :crit,    a8, 2026-07-15, 14d
```

The flow of operations looks like this:

```
flowchart TD
    subgraph New_Order_Arrival [New Order Arrival]
        N[Order Created] --> A{Drivers Available?}
        A -->|No| WAIT[Add to Pending Queue]
        A -->|Yes| B[Compute Insertion Cost per Driver]
        B --> C{Choose Min Cost Driver}
        C --> D[Insert into that driver's route]
        D --> E[Optimize route (2-opt)]
        E --> F[Save route & ETA]
    end
    subgraph Driver_Pickup [Driver Pickup]
        P[Driver picks up Order] --> Q[Mark Active in DB]
        Q --> R{Re-run Optimization?}
        R -->|Yes| E
        R -->|No| S[Continue]
    end
```

This ensures each new order is assigned optimally on the fly, and routes are continually refined.

References

The above design is grounded in authoritative sources: OSRM's documentation ¹² and GraphHopper's performance report ⁵ for matrix services, Mapbox API limits ⁶ for planning limits, and algorithmic research on dynamic VRP and local search ² ³ ¹⁰. Each recommendation is chosen for proven speed and practicality in real-time logistics.

¹ Held-Karp algorithm - Wikipedia

https://en.wikipedia.org/wiki/Held%E2%80%93Karp_algorithm

² optimization-online.org

<https://optimization-online.org/wp-content/uploads/2022/11/dynamic-insertion.pdf>

³ 2-opt - Wikipedia

<https://en.wikipedia.org/wiki/2-opt>

⁴ How to Build a Free, Scalable Driving Distance Matrix Generator with Python and OSRM | by Mehmet Karahan Çetinkaya | Medium

<https://medium.com/@mehmetkarahanc/how-to-build-a-free-scalable-driving-distance-matrix-generator-with-python-and-osrm-99a4d52442fc>

⁵ Matrix API Update: 180 000 road distances per second - GraphHopper Directions API

<https://www.graphhopper.com/blog/2018/03/22/matrix-api-update-180k-road-distances-per-second/>

⁶ Matrix API | API Docs | Mapbox

<https://docs.mapbox.com/api/navigation/matrix/>

⁷ Transactions - Database Manual - MongoDB Docs

<https://www.mongodb.com/docs/manual/core/transactions/>

⁸ proceedings.mlr.press

<https://proceedings.mlr.press/v157/sui21a/sui21a.pdf>

⁹ ¹⁰ ¹¹ Routing Options | OR-Tools | Google for Developers

https://developers.google.com/optimization/routing/routing_options

¹² OSRM API Documentation

<https://project-osrm.org/docs/v26.4.0/index.html>